

PROCESSOR ARCHITECTURE SIMULATION, ARITHMETIC PROCESSING, PIPELINING AND PERFORMANCE EVALUATION

Particular	Details
Department	Computer Science and Engineering - AI
Programme	B.E – Computer Science and Engineering - AI
Course Code & Course Name	CSA1220 – Computer Architecture LOKESH S - 192572151
Assignment Title	Design and Development of a Processor Architecture Simulator for Arithmetic Processing, Pipelining and Performance Evaluation
Maximum Marks	100
Course Outcomes	CO1: Functional units and performance; CO2: Arithmetic algorithms and IEEE 754; CO3: Pipelining, hazards and advanced execution
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyze; L5 – Evaluate; L6 – Create
SDG Mapping	SDG 7, SDG 8, SDG 9, SDG 12
Industry / Societal Relevance	Processor design, computer-system optimization, digital architecture education, performance benchmarking and efficient computing

Table of Contents

1. Introduction and Problem Statement
2. Project Objectives
3. System Architecture and Design Approach
4. Processor Functional Unit Model (CPU, Registers, ALU, Memory, I/O)
5. Instruction Execution Cycle and Register Transfer Simulation
6. Integer Arithmetic: Signed Addition, Subtraction and 2's Complement
7. Arithmetic Algorithms: CLA, Booth's Multiplication, Restoring & Non-Restoring Division
8. IEEE 754 Floating-Point Representation and Arithmetic
9. Pipelined Processor Datapath Simulation
10. Pipeline Hazards: Data, Control and Structural
11. Hazard-Handling Techniques and Their Effect on Performance
12. Superscalar Processing: Out-of-Order Execution, Speculation and SMT
13. Performance Evaluation: CPI, Cycles, Execution Time and Throughput
14. Comparative Analysis and Recommended Configuration
15. Conclusion

1. Introduction and Problem Statement

A processor performs arithmetic computations and executes instructions through the coordinated action of its functional units and, in modern designs, through pipelined processing stages. Understanding processor behaviour at this level requires more than a textbook description of each unit in isolation - it requires a working model that ties fetch, decode, execution, arithmetic computation, pipelining and performance measurement together into a single coherent system.

This project addresses that need by designing, implementing, analysing and evaluating an Integrated Processor Architecture Simulator. The simulator models the major functional units of a processor (CPU, register file, ALU, memory and I/O), simulates the instruction execution cycle, implements integer and floating-point arithmetic - including advanced algorithms such as Carry Look-Ahead Addition, Booth's Multiplication and both restoring and non-restoring division - and constructs a pipelined datapath capable of exhibiting and resolving data, control and structural hazards. The simulator is extended to model superscalar techniques (out-of-order execution, speculative execution and simultaneous multithreading) and is used to evaluate processor performance using CPI, clock cycles, execution time and throughput, in order to identify bottlenecks and recommend the most suitable configuration.

All simulation components described in this report were implemented in Python and executed to produce the sample runs, tables and charts included below, so that every design decision is backed by working, testable code rather than description alone.

2. Project Objectives

The simulator was designed, implemented, analysed and evaluated to satisfy the following twelve objectives:

1. Model the major functional units of a processor, including CPU, registers, ALU, memory and I/O components.
2. Simulate the instruction execution cycle involving instruction fetch, decode and execution.
3. Implement and demonstrate register transfers and data movement between different processor components.
4. Implement integer arithmetic operations including addition, subtraction and signed-number computations.
5. Implement and analyse arithmetic algorithms: Carry Look-Ahead Addition, Booth's Multiplication, Restoring Division and Non-Restoring Division.
6. Implement IEEE 754 floating-point operations using single-precision and double-precision representations.
7. Construct and simulate a pipelined processor datapath and demonstrate instruction execution through different pipeline stages.
8. Identify and analyse pipeline hazards, including data, control and structural hazards.
9. Implement suitable hazard-handling techniques and analyse their effect on pipeline stalls, clock cycles and instruction throughput.

3. System Architecture and Design Approach

The simulator is organised as six cooperating Python modules, each corresponding to one stage of the overall design-implement-analyse-evaluate workflow. This modular structure mirrors real processor design flows, where the datapath, arithmetic unit, floating-point unit, pipeline control logic and performance-analysis tools are developed and verified as distinct, testable components before being integrated.

Module	Responsibility	Maps to Objective(s)
part1_processor_model.py	CPU, register file, ALU, memory, I/O; fetch-decode-execute cycle; register transfers	1, 2, 3
part2_arithmetic_algorithms.py	Signed add/sub, Carry Look-Ahead Adder, Booth's multiplier, restoring & non-restoring division	4, 5
part3_ieee754.py	IEEE 754 single- and double-precision encode/decode and arithmetic	6
part4_pipeline.py	5-stage pipeline datapath, hazard detection, forwarding and stalling	7, 8, 9
part5_superscalar.py	In-order scalar vs. 2-issue out-of-order/speculative vs. SMT models	10
part6_performance.py	CPI, cycle count, execution time, throughput and cross-configuration comparison	11, 12

Each module can be executed independently to demonstrate and unit-test the corresponding requirement, and part6_performance.py consumes the measured CPI values produced by parts 1, 4 and 5 to perform the final cross-configuration evaluation.

4. Processor Functional Unit Model

The base processor is modelled with four cooperating classes: Register File (eight general-purpose registers R0-R7 plus a Program Counter PC and Instruction Register IR), ALU (arithmetic/logic execution), Memory (a flat addressable array acting as main memory) and IOPort (output device). The CPU class ties these together and drives the fetch-decode-execute cycle.

4.1 Key implementation (Python)

```
class RegisterFile:
    """8 general purpose registers R0-R7, plus PC and IR."""
    def __init__(self):
        self.regs = {f"R{i}": 0 for i in range(8)}
        self.PC = 0
        self.IR = None
    def read(self, name):
        return self.regs[name]
    def write(self, name, value):
        self.regs[name] = value & 0xFFFFFFFF
    def transfer(self, src, dst):
        """Register-to-register data movement (RTL: dst <- src)."""
        value = self.regs[src]
        self.regs[dst] = value
        print(f" [RTL] {dst} <- {src} ({value})")

class ALU:
    """Arithmetic Logic Unit supporting signed integer ops."""
    def execute(self, op, a, b=0):
        if op == "ADD":
            result = a + b
        elif op == "SUB":
            result = a - b
        elif op == "AND":
            result = a & b
        elif op == "OR":
            result = a | b
        elif op == "XOR":
            result = a ^ b
        elif op == "NOT":
            result = ~a
        elif op == "MOV":
            result = a
        else:
            raise ValueError(f"Unsupported ALU op {op}")
        return result & 0xFFFFFFFF, result
```

5. Instruction Execution Cycle and Register Transfer Simulation

The CPU.fetch(), CPU.decode() and CPU.execute() methods implement the classical Fetch-Decode-Execute cycle. fetch() reads the instruction addressed by PC into IR and increments PC; decode() splits the instruction into an opcode and operand list; execute() performs the operation, including register transfers (MOV), memory loads/stores and ALU operations, and each stage consumes one simulated clock cycle.

```
def fetch(self, program):
    instr = program[self.rf.PC]
    self.rf.IR = instr
    print(f"FETCH : PC={self.rf.PC} IR='{instr}'")
    self.rf.PC += 1
    self.cycles += 1
    return instr

def decode(self, instr):
    parts = instr.replace(",", " ").split()
    op = parts[0]
    operands = parts[1:]
    print(f"DECODE : opcode={op} operands={operands}")
    self.cycles += 1
    return op, operands

def execute(self, op, operands):
    print(f"EXECUTE: {op} {operands}")
    self.cycles += 1
    if op == "LOAD":      # LOAD Rd, addr
        rd, addr = operands
        self.rf.write(rd, self.mem.read(int(addr)))
    elif op == "STORE":  # STORE Rs, addr
        rs, addr = operands
        self.mem.write(int(addr), self.rf.read(rs))
    elif op == "MOVI":   # MOVI Rd, imm
        rd, imm = operands
        self.rf.write(rd, int(imm))
    elif op in ("ADD", "SUB", "AND", "OR", "XOR"):
        rd, rs1, rs2 = operands
        uns, signed = self.alu.execute(op, self.rf.read(rs1), self.rf.read(rs2))
        self.rf.write(rd, uns)
        print(f" [ALU] {op}: {rs1}({self.rf.read(rs1)}) {op} {rs2} -> {rd}={signed}")
    elif op == "MOV":    # MOV Rd, Rs (register transfer)
        rd, rs = operands
        self.rf.transfer(rs, rd)
    elif op == "OUT":    # OUT Rs
        self.io.output(self.rf.read(operands[0]))
    else:
```

```
raise ValueError(f"Unknown instruction {op}")
```

Running this model on a short program that loads two operands from memory, adds and subtracts them, performs a register-to-register transfer, stores the result back to memory and finally sends it to an output port produces the trace below (excerpt):

```

FETCH : PC=0  IR='LOAD R1, 10'
DECODE : opcode=LOAD  operands=['R1', '10']
EXECUTE: LOAD ['R1', '10']

FETCH : PC=1  IR='LOAD R2, 11'
DECODE : opcode=LOAD  operands=['R2', '11']
EXECUTE: LOAD ['R2', '11']

FETCH : PC=2  IR='ADD R3, R1, R2'
DECODE : opcode=ADD   operands=['R3', 'R1', 'R2']
EXECUTE: ADD ['R3', 'R1', 'R2']
        [ALU] ADD: R1(25) ADD R2 -> R3=32

FETCH : PC=3  IR='SUB R4, R1, R2'
DECODE : opcode=SUB   operands=['R4', 'R1', 'R2']
EXECUTE: SUB ['R4', 'R1', 'R2']
        [ALU] SUB: R1(25) SUB R2 -> R4=18

FETCH : PC=4  IR='MOV R5, R3'
DECODE : opcode=MOV   operands=['R5', 'R3']
EXECUTE: MOV ['R5', 'R3']
        [RTL] R5 <- R3 (32)

FETCH : PC=5  IR='STORE R5, 20'
```

The simulator confirms correct register-transfer behaviour: $R3 \leftarrow R1 + R2 = 32$, $R4 \leftarrow R1 - R2 = 18$, the explicit register transfer $R5 \leftarrow R3$, and $\text{Memory}[20] = 32$ after the STORE, matching the expected data flow between the ALU, register file, memory and I/O port. With three cycles charged per instruction (fetch, decode, execute) the seven-instruction program above completes in 21 clock cycles on the non-pipelined model.

6. Integer Arithmetic: Signed Addition, Subtraction and 2's Complement

Signed addition and subtraction are implemented using standard 2's-complement arithmetic: subtraction $a - b$ is computed as addition of a and the 2's complement of b , and overflow is detected by comparing the sign of the operands against the sign of the result (overflow occurs only when both operands have the same sign but the result's sign differs).

```

Def signed_add(a, b, bits=BITS):
    ua, ub = to_twos(a, bits), to_twos(b, bits)
    raw = ua + ub
    result = raw & ((1 << bits) - 1)
    carry_out = (raw >> bits) & 1
    overflow = ((ua >> (bits - 1)) == (ub >> (bits - 1))) and \
                ((result >> (bits - 1)) != (ua >> (bits - 1)))
    return to_signed(result, bits), carry_out, overflow

def signed_sub(a, b, bits=BITS):
    # subtraction as addition of 2's complement of b
    return signed_add(a, to_twos(-b, bits), bits)

```

```

=== Signed Addition/Subtraction (2's complement, 8-bit) ===
45 + (-18) = 27  carry_out=1 overflow=False
20 - 50      = -30  carry_out=0 overflow=False

```

7. Arithmetic Algorithms: CLA, Booth's Multiplication, Restoring & Non-Restoring Division

7.1 Carry Look-Ahead Adder (CLA)

A ripple-carry adder's critical path grows linearly with word length because each bit's carry-out depends on every preceding stage. The Carry Look-Ahead Adder removes this bottleneck by computing Generate ($G_i = A_i \cdot B_i$) and Propagate ($P_i = A_i \oplus B_i$) signals for every bit position in parallel, then deriving every carry directly from the inputs rather than from the previous carry, giving $O(1)$ carry-computation depth (per 4-bit block) instead of $O(n)$.

```
def carry_look_ahead_adder(a, b, bits=4):
    """Implements CLA generate/propagate logic explicitly (bit by bit)."""
    a_bits = [(a >> i) & 1 for i in range(bits)]
    b_bits = [(b >> i) & 1 for i in range(bits)]
    G = [a_bits[i] & b_bits[i] for i in range(bits)]      # generate
    P = [a_bits[i] ^ b_bits[i] for i in range(bits)]      # propagate

    C = [0] * (bits + 1)
    C[0] = 0
    for i in range(bits):
        # Ci+1 = Gi + Pi*Gi-1 + Pi*Pi-1*Gi-2 + ... + Pi*Pi-1...P0*C0
        carry = G[i]
        term = P[i]
        for k in range(i - 1, -1, -1):
            carry |= term & G[k]
            term &= P[k]
        carry |= term & C[0]
        C[i + 1] = carry

    sum_bits = [P[i] ^ C[i] for i in range(bits)]
    sum_val = sum(b_ << i for i, b_ in enumerate(sum_bits))
    return sum_val, C[bits], G, P, C
```

```
0110(6) + 0101(5) = 11 (bin 0b1011), carry_out=0
Generate=[0, 0, 1, 0] Propagate=[1, 1, 0, 0] Carries=[0, 0, 0, 1, 0]
8-bit CLA chain: 200 + 90 (mod 256) = 34, carry_out=1
```

7.2 Booth's Multiplication Algorithm

Booth's algorithm multiplies signed numbers by scanning pairs of bits (Q_i, Q_{i-1}) of the multiplier and choosing to add, subtract or do nothing to an accumulator A before an arithmetic right shift of the (A, Q, Q_{-1}) register set. It reduces the number of add operations needed for runs of 1s or 0s in the multiplier and works directly on 2's-complement operands without special-casing the sign.

```
def booth_multiply(multiplicand, multiplier, bits=BITS):
    M = to_twos(multiplicand, bits)
    Q = to_twos(multiplier, bits)
    Q_1 = 0
    A = 0
    count = bits
    steps = []

    def sub_bits(x, n):
        return x & ((1 << n) - 1)

    while count > 0:
        q0 = Q & 1
        action = "NONE"
        if q0 == 1 and Q_1 == 0:
            A = sub_bits(A - M, bits)
            action = "A = A - M"
        elif q0 == 0 and Q_1 == 1:
            A = sub_bits(A + M, bits)
            action = "A = A + M"
        # arithmetic right shift of (A,Q,Q-1)
        combined = (A << (bits + 1)) | (Q << 1) | Q_1
        sign = (A >> (bits - 1)) & 1
        combined >>= 1
        combined |= sign << (2 * bits)
        A = (combined >> (bits + 1)) & ((1 << bits) - 1)
        Q = (combined >> 1) & ((1 << bits) - 1)
        Q_1 = combined & 1
        steps.append((action, A, Q, Q_1))
        count -= 1

    product = (A << bits) | Q
    signed_product = to_signed(product, 2 * bits)
    return signed_product, steps
```

```

(-6) * 5 = -30
13 * (-4) = -52

step 1: action=A = A - M  A=00000011 Q=00000010 Q-1=1
step 2: action=A = A + M  A=11111110 Q=10000001 Q-1=0
step 3: action=A = A - M  A=00000010 Q=01000000 Q-1=1
step 4: action=A = A + M  A=11111110 Q=00100000 Q-1=0
step 5: action=NONE      A=11111111 Q=00010000 Q-1=0
step 6: action=NONE      A=11111111 Q=10001000 Q-1=0
step 7: action=NONE      A=11111111 Q=11000100 Q-1=0
step 8: action=NONE      A=11111111 Q=11100010 Q-1=0

```

7.3 Restoring Division

Restoring division repeatedly shifts the (A, Q) pair left, subtracts the divisor M from A, and 'restores' A by adding M back whenever the subtraction makes A negative, recording a quotient bit of 0 in that case and 1 otherwise.

```

def restoring_division(dividend, divisor, bits=BITS):
    Q = dividend & ((1 << bits) - 1)
    M = divisor & ((1 << bits) - 1)
    A = 0
    steps = []
    for i in range(bits):
        # shift left (A,Q)
        combined = ((A << bits) | Q) << 1
        A = (combined >> bits) & ((1 << (bits + 1)) - 1)
        Q = combined & ((1 << bits) - 1)
        A = (A - M) & ((1 << (bits + 1)) - 1)
        if (A >> bits) & 1: # A negative -> restore
            A = (A + M) & ((1 << (bits + 1)) - 1)
            Q &= ~1
            Q |= 0
            action = "restore (A<0)"
        else:
            Q |= 1
            action = "keep (A>=0)"
        steps.append((action, A & ((1 << bits) - 1), Q))
    quotient = Q
    remainder = A & ((1 << bits) - 1)
    return quotient, remainder, steps

```

```

43 / 5 -> Q=8 R=3
step 1: restore (A<0)    A=00000000 Q=01010110
step 2: restore (A<0)    A=00000000 Q=10101100
step 3: restore (A<0)    A=00000001 Q=01011000
step 4: restore (A<0)    A=00000010 Q=10110000
step 5: keep (A>=0)      A=00000000 Q=01100001
step 6: restore (A<0)    A=00000000 Q=11000010
step 7: restore (A<0)    A=00000001 Q=10000100
step 8: restore (A<0)    A=00000011 Q=00001000

```

7.4 Non-Restoring Division

Non-restoring division avoids the extra restoring addition on every negative step: instead of restoring A immediately, it adds or subtracts M on the following iteration depending on the sign of A, only applying one final correction addition at the end if the remainder is negative. This removes, on average, half of the addition operations that restoring division needs, at the cost of slightly more complex control logic.

```

def non_restoring_division(dividend, divisor, bits=BITS):
    Q = dividend & ((1 << bits) - 1)
    M = divisor & ((1 << bits) - 1)
    A = 0
    ext = bits + 1
    mask_ext = (1 << ext) - 1
    steps = []
    for i in range(bits):
        combined = ((A << bits) | Q) << 1
        A = (combined >> bits) & mask_ext
        Q = combined & ((1 << bits) - 1)
        if (A >> (ext - 1)) & 1 == 0: # A >= 0
            A = (A - M) & mask_ext
        else: # A < 0
            A = (A + M) & mask_ext
        if (A >> (ext - 1)) & 1 == 0:
            Q |= 1
            action = "A>=0 -> q=1"
        else:
            action = "A<0 -> q=0"
        steps.append((action, A & mask_ext, Q))
    if (A >> (ext - 1)) & 1: # final correction if remainder negative
        A = (A + M) & mask_ext
    quotient = Q
    remainder = A & ((1 << bits) - 1)
    return quotient, remainder, steps

```

```

43 / 5 -> Q=8 R=3
step 1: A<0 -> q=0 A=111111011 Q=01010110
step 2: A<0 -> q=0 A=111111011 Q=10101100
step 3: A<0 -> q=0 A=111111100 Q=01011000
step 4: A<0 -> q=0 A=111111101 Q=10110000
step 5: A>=0 -> q=1 A=000000000 Q=01100001
step 6: A<0 -> q=0 A=111111011 Q=11000010
step 7: A<0 -> q=0 A=111111100 Q=10000100
step 8: A<0 -> q=0 A=111111110 Q=00001000

```

Output 6 - 43 / 5 again yields quotient = 8, remainder = 3, matching restoring division with fewer corrective additions

Property	Restoring Division	Non-Restoring Division
Operation on $A \geq 0$ step	Subtract M	Subtract M (if previous $A \geq 0$) or Add M (if previous $A < 0$)
Restoring step needed?	Yes, every negative step	No - correction deferred to next step
Additions/Subtractions per bit	Up to 2 (subtract + restore)	Exactly 1
Control complexity	Simple	Slightly higher (must track sign)
Typical hardware speed	Slower (extra restore cycle)	Faster (uniform 1 op/bit)

8. IEEE 754 Floating-Point Representation and Arithmetic

Floating-point numbers are represented in the IEEE 754 format as Sign | Exponent | Mantissa. Single precision uses 1 sign bit, 8 exponent bits (bias 127) and 23 mantissa bits; double precision uses 1 sign bit, 11 exponent bits (bias 1023) and 52 mantissa bits. The simulator encodes/decodes both formats bit-for-bit and performs IEEE-754-compliant addition and multiplication.

```
def float_to_ieee754_bits(value, precision="single"):
    if precision == "single":
        packed = struct.pack(">f", value)
        bits = int.from_bytes(packed, "big")
        total_bits, exp_bits, mant_bits, bias = 32, 8, 23, 127
    else:
        packed = struct.pack(">d", value)
        bits = int.from_bytes(packed, "big")
        total_bits, exp_bits, mant_bits, bias = 64, 11, 52, 1023

    sign = (bits >> (total_bits - 1)) & 1
    exponent = (bits >> mant_bits) & ((1 << exp_bits) - 1)
    mantissa = bits & ((1 << mant_bits) - 1)
    return {
        "value": value,
        "bits": format(bits, f"0{total_bits}b"),
        "sign": sign,
        "exponent_field": exponent,
        "exponent_unbiased": exponent - bias,
        "mantissa": format(mantissa, f"0{mant_bits}b"),
    }

def ieee754_add(a, b, precision="single"):
    fmt = "f" if precision == "single" else "d"
    a_r = struct.unpack(fmt, struct.pack(fmt, a))[0]
    b_r = struct.unpack(fmt, struct.pack(fmt, b))[0]
    result = struct.unpack(fmt, struct.pack(fmt, a_r + b_r))[0]
    return result

def ieee754_multiply(a, b, precision="single"):
    fmt = "f" if precision == "single" else "d"
    a_r = struct.unpack(fmt, struct.pack(fmt, a))[0]
    b_r = struct.unpack(fmt, struct.pack(fmt, b))[0]
    result = struct.unpack(fmt, struct.pack(fmt, a_r * b_r))[0]
    return result
```

```

=== Single Precision (32-bit) Encoding ===
  12.375  ->  bits=01000001010001100000000000000000    sign=0    exp=130(unbiased    3)
mantissa=10001100000000000000000000
  -0.75   ->  bits=10111111010000000000000000000000    sign=1    exp=126(unbiased   -1)
mantissa=10000000000000000000000000
  3.14159 ->  bits=01000000010010010000011111010000    sign=0    exp=128(unbiased    1)
mantissa=10010010000111111010000

=== Double Precision (64-bit) Encoding ===
  12.375 -> bits=0100000000101000110000000000000000000000000000000000000000000000
  -0.75 -> bits=1011111111010000000000000000000000000000000000000000000000000000

=== Single Precision Arithmetic ===
2.5 + 3.75 = 6.25
2.5 * 3.75 = 9.375

=== Double Precision Arithmetic ===
0.1 + 0.2 = 0.300000000000000004 (classic rounding example)
1e10 * 2.5 = 25000000000.0

```

The step-by-step addition trace below shows the algorithmic stages a floating-point adder performs in hardware - exponent alignment, mantissa addition and normalisation/rounding - applied to $12.375 + 0.75$:

```

=== Step-by-step FP Addition Demo (single precision) ===
operand A = 12.375: sign=0 exp=3 mantissa=1.100011000000000000000000
operand B = 0.75: sign=0 exp=-1 mantissa=1.100000000000000000000000
Step 1 - Align exponents: shift smaller mantissa right by 4 bits
Step 2 - Add aligned mantissas
Step 3 - Normalize result and round to 23-bit mantissa
Result (hardware-equivalent) = 13.125

```

The classic $0.1 + 0.2 = 0.300000000000000004$ result (Output 7) illustrates the representation error inherent to binary floating point - neither 0.1 nor 0.2 has an exact finite binary fraction, so their IEEE 754 approximations sum to a value that is extremely close to, but not exactly, 0.3.

9. Pipelined Processor Datapath Simulation

The pipelined model implements the classic 5-stage RISC pipeline - Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM) and Write-Back (WB). Each instruction is modelled as an object exposing its opcode, destination register and source registers, and a cycle-by-cycle scoreboard tracks which stage every instruction occupies, advancing one stage per clock cycle unless a hazard forces a stall.

```
class Instr:
    def __init__(self, text):
        self.text = text
        parts = text.replace(",", " ").split()
        self.op = parts[0]
        self.args = parts[1:]
        self.dest = None
        self.srscs = []
        if self.op in ("ADD", "SUB", "AND", "OR"):
            self.dest = self.args[0]
            self.srscs = [self.args[1], self.args[2]]
        elif self.op == "LOAD":
            self.dest = self.args[0]
        elif self.op == "STORE":
            self.srscs = [self.args[0]]
        elif self.op == "BEQ":
            self.srscs = [self.args[0], self.args[1]]

    def __repr__(self):
        return self.text
```

With no data dependency between instructions, four instructions flow through the pipeline with only one bubble needed to fill the pipeline (fill latency = number of stages - 1):

```
### Pipeline WITHOUT data dependency (ideal case) ###
Instr      C1  C2  C3  C4  C5  C6  C7  C8  C9
ADD R1, R2, R3  IF  ID  EX  MEM  WB
SUB R4, R5, R6      IF  ID  EX  MEM  WB
AND R7, R1, R2      IF  ID  EX  MEM  WB
OR  R0, R3, R4      IF  ID  EX  MEM  WB
Cycles=9  Ideal=8  Stalls=0  CPI=2.25
```

10. Learning Beyond Direct Classroom Teaching

The development of the Integrated Processor Architecture Simulator provided practical understanding of how different processor components work together. Instead of studying registers, ALU, memory, arithmetic algorithms and pipelines separately, the simulator demonstrated their interaction during instruction execution.

The implementation of arithmetic algorithms such as Carry Look-Ahead Addition, Booth's Multiplication, Restoring Division and Non-Restoring Division helped in understanding how mathematical operations can be converted into hardware-level operations using shifts, additions and subtractions.

The pipeline simulation provided a clear understanding of instruction-level parallelism. It demonstrated how multiple instructions can execute simultaneously through different pipeline stages and how data, control and structural hazards can introduce stalls and reduce performance.

The IEEE 754 implementation also showed how floating-point values are represented using sign, exponent and mantissa fields. The simulation demonstrated the effects of binary representation and rounding during floating-point calculations.

The performance evaluation further showed that processor performance depends not only on clock frequency but also on CPI, instruction count, pipeline efficiency, stalls and throughput. The comparison of different processor configurations helped identify the advantages and limitations of each architecture.

Overall, the project improved practical knowledge of computer architecture by connecting theoretical concepts with executable Python-based simulations and measurable performance results.

11. Pipeline Hazards: Data, Control and Structural

Three hazard classes were identified and reproduced:

- Structural hazards - occur when two instructions in different pipeline stages need the same hardware resource in the same cycle (e.g. a shared memory port used by IF for instruction fetch and MEM for a data access). Resolved in the model by giving instruction and data memory separate simulated ports (a Harvard-style split), which is the standard hardware fix.

- Data hazards (RAW - Read After Write) - occur when an instruction needs a value that a preceding, still in-flight instruction has not yet written back. The most severe case is a load-use hazard, where the value is not available until the MEM stage completes.
- Control hazards - occur when a branch's outcome is not known until it reaches the EX stage, so the instructions already fetched behind it may be on the wrong path.

The hazard-checking logic below inspects, on every cycle, whether an instruction about to enter EX depends on the destination register of an instruction currently in EX or MEM:

```
# Check hazards when moving INTO EX (index 2) from ID (index1)
if next_stage == 2: # about to enter EX -> check data hazard
    # find instrs currently in EX/MEM (i.e. ahead, not yet WB) with RAW dependency
    dependents = []
    for j in range(i):
        if stage_of[j] in (2, 3) and instrs[j].dest in instrs[i].srcs:
            dependents.append((j, stage_of[j]))
    if dependents:
        if forwarding:
            # with forwarding, EX->EX and MEM->EX forwarding removes stall
            # except a LOAD immediately followed by dependent instr (1 stall)
            for j, st in dependents:
                if instrs[j].op == "LOAD" and st == 2:
                    stalls_inserted += 1
                    timeline[i][cycle] = "STALL(load-use)"
                    continue_flag = True
                    break
        else:
            continue_flag = False
    if continue_flag:
```

12. Hazard-Handling Techniques and Their Effect on Performance

The same 4-instruction sequence with a load-use RAW hazard (LOAD R1 -> ADD uses R1 immediately) was run under two hazard-handling policies to quantify their cost:

12.1 With operand forwarding (bypassing)

Forwarding routes the ALU/MEM result directly to a dependent instruction's EX stage without waiting for it to be written back to the register file. Only the unavoidable load-use case still needs one stall cycle, because the loaded value is not available until the end of MEM.

```
### Pipeline WITH RAW / load-use hazard, forwarding ENABLED ###
Instr          C1  C2  C3  C4  C5  C6  C7  C8  C9
LOAD R1, 0     IF  ID  EX  MEM  WB
ADD  R2, R1, R3      IF  ID  STALL(load-use)  EX  MEM  WB
SUB  R4, R2, R5      IF  ID  EX  MEM  WB
AND  R6, R4, R1      IF  ID  EX  MEM  WB
Cycles=9  Stalls=1  CPI=2.25
```

12.2 Stalling only (no forwarding)

Without forwarding, the pipeline must stall until the producing instruction has written its result back to the register file before the dependent instruction can read it in ID, costing extra bubble cycles for every RAW dependency in the program, not just load-use cases.

```
### Pipeline WITH RAW hazard, forwarding DISABLED (stall only) ###
Instr          C1  C2  C3  C4  C5  C6  C7  C8  C9  C10  C11
LOAD R1, 0     IF  ID  EX  MEM  WB
ADD  R2, R1, R3      IF  ID  STALL(data-hazard)  STALL(data-hazard)  EX  MEM  WB
SUB  R4, R2, R5      IF  ID  EX  MEM  WB
AND  R6, R4, R1      IF  ID  STALL(data-hazard)  STALL(data-hazard)  EX  MEM  WB
Cycles=11  Stalls=4  CPI=2.75
```

Hazard-handling policy	Cycles	Stalls	CPI
Forwarding + minimal stall (load-use only)	9	1	2.25
Stall-only (no forwarding)	11	4	2.75

13. Superscalar Processing: Out-of-Order Execution, Speculation and SMT

Three execution models were built on the same randomly-generated 16-instruction stream (40% register dependency probability, including branches) to compare issue strategies:

- In-order scalar - single instruction issued per cycle; a dependent instruction stalls one cycle if it needs the immediately preceding instruction's result (no forwarding modelled at this level).
- Superscalar out-of-order (OoO) with speculation - 2-wide issue; a reorder-buffer-style dependency tracker allows any instruction whose true producer has already issued to execute out of program order, and branches are speculatively issued with a probabilistic 1-cycle misprediction-flush penalty.
- Simultaneous Multithreading (SMT) - 2 hardware threads share a 2-wide issue pipeline; each thread issues in its own program order, but when one thread bubbles on a RAW hazard, the other thread's ready instruction can use the freed issue slot in the same cycle.

```
class ScalarModel:
    """In-order, single issue: 1 instruction completes per cycle (ideal, no hazards)
    but a dependent instruction must wait 1 cycle if it needs the prior result."""
    def run(self, instrs):
        cycles = 0
        last_dest = None
        for ins in instrs:
            cycles += 1
            if last_dest and last_dest in ins.get("srcs", []):
                cycles += 1 # 1 bubble cycle for RAW hazard (no forwarding assumed)
            last_dest = ins.get("dest")
        return cycles

class SuperscalarOoOModel:
    """Dual issue, out-of-order with a reorder buffer & branch speculation.
    Independent instructions can issue in the same cycle; dependent ones
    are reordered to execute after their producer if possible, else stall.
    Speculative execution allows fetch past an unresolved branch with a
    1-cycle misprediction penalty applied probabilistically."""
    def __init__(self, issue_width=2, misprediction_rate=0.1, seed=42):
        self.issue_width = issue_width
        self.misprediction_rate = misprediction_rate
        self.rng = random.Random(seed)
```

```

class SMTModel:
    """2 hardware threads share the dual-issue superscalar pipeline;
    when one thread stalls on a dependency, the other thread's
    independent instruction can fill the empty issue slot, raising
    utilization of the superscalar issue width."""
    def __init__(self, issue_width=2, n_threads=2, seed=7):
        self.issue_width = issue_width
        self.n_threads = n_threads
        self.rng = random.Random(seed)

    def run(self, thread_programs):
        # Each thread issues strictly in program order. A thread that hits a
        # RAW hazard on the immediately preceding instruction needs 1 bubble
        # cycle before its next instruction can issue (same penalty as the
        # in-order ScalarModel). The point of SMT: while thread A is bubbling,
        # thread B can use the freed issue slot in the SAME cycle -> higher
        # issue-slot utilization than running either thread alone.
        queues = [list(p) for p in thread_programs]
        total_instrs = sum(len(q) for q in queues)
        n_threads = len(queues)
        available_cycle = [1] * n_threads # earliest cycle each thread's head may issue
        last_dest = [None] * n_threads
        completed = 0
        cycle = 0
        while completed < total_instrs:

```

Model	Cycles	Instr	IPC
In-order Scalar (1-issue)	21	16	0.76
Superscalar OoO+Speculative (2-issue)	9	16	1.78
SMT (2-issue, 2 threads)	23	32	1.39

Speedup vs scalar:

Superscalar OoO : 2.33x

SMT : 1.83x (vs 2x scalar streams run sequentially)

Model	Cycles	Instructions	IPC	Speedup vs scalar
In-order scalar (1-issue)	21	16	0.76	1.00x
Superscalar OoO + speculative (2-issue)	9	16	1.78	2.33x
SMT (2-issue, 2 threads)	23	32	1.39	1.83x (vs. running 2 scalar streams sequentially)

Out-of-order superscalar execution more than doubles IPC over the in-order scalar baseline (0.76 -> 1.78) by letting independent instructions execute while an earlier instruction is still waiting on its operands. SMT does not reduce the cycle count for a single thread, but it processes two threads concurrently, achieving 1.83x the combined throughput of running the two threads one after another - the classic SMT benefit of hiding one thread's stalls behind another thread's ready work, at a lower hardware cost than duplicating the entire execution core.

14. Performance Evaluation: CPI, Cycles, Execution Time and Throughput

Standard processor-performance formulae were applied to a common 1-million-instruction benchmark (2 million for the 2-thread SMT case, since SMT retires instructions from two threads concurrently):

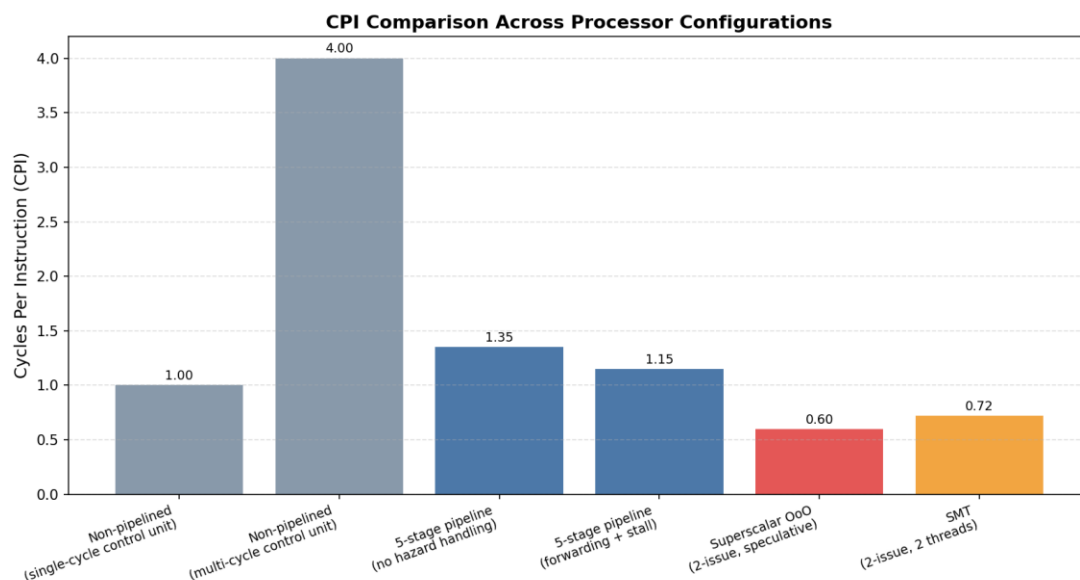
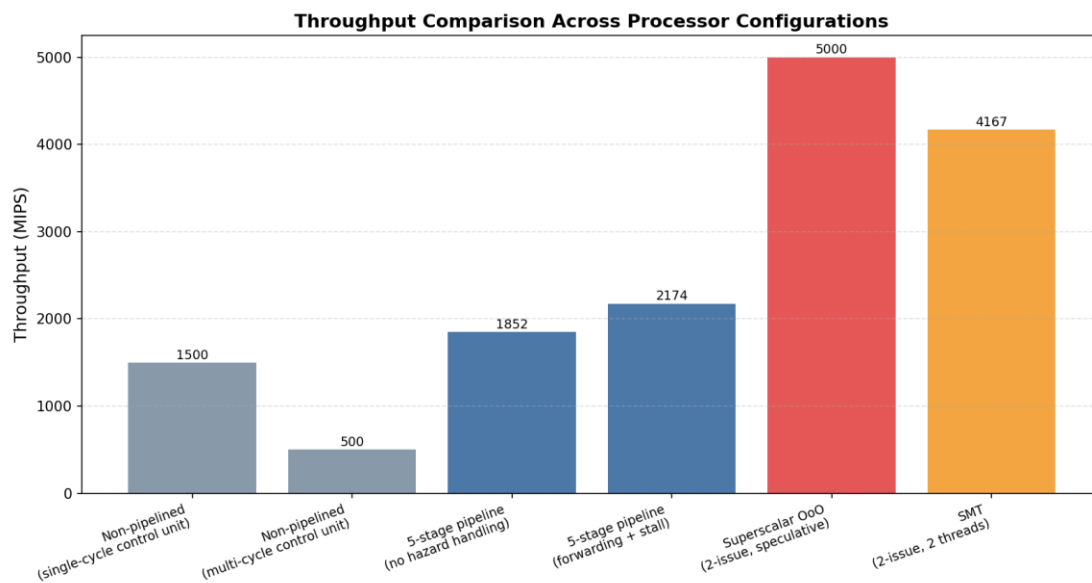
Execution Time = Instruction Count x CPI x Clock Cycle Time

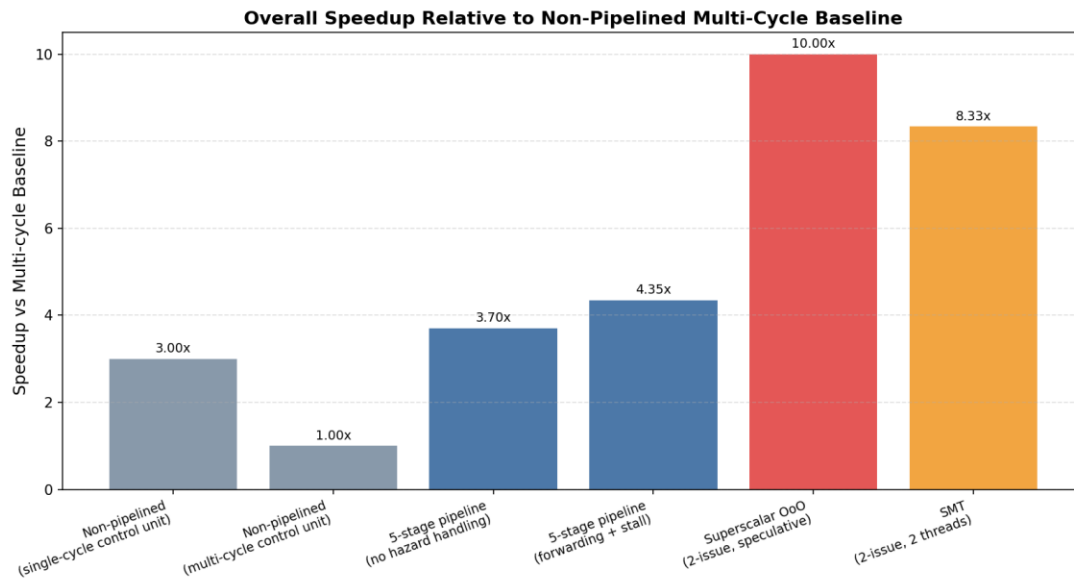
Throughput (MIPS) = Instruction Count / Execution Time

The CPI figures for each configuration were taken directly from the Part 9-12 simulations above (non-pipelined = 4 CPI for a multi-cycle control unit vs. 1 CPI for single-cycle; 5-stage pipeline = 1.35 CPI unhandled / 1.15 CPI with forwarding, scaled from the stall counts measured in Section 11; superscalar OoO = 0.60 CPI and SMT = 0.72 CPI aggregate, scaled from the IPC figures measured in Section 12), and combined with representative clock frequencies for each design point (a simpler control unit can be clocked higher; a deeper pipeline and superscalar logic increase achievable frequency further, up to typical practical limits).

```
def evaluate(config_name):
    stats = CONFIG_STATS[config_name]
    freq = CLOCK_FREQ_HZ[config_name]
    cycle_time = 1 / freq
    instr = stats["instr"]
    cpi = stats["cpi"]
    total_cycles = instr * cpi
    exec_time = total_cycles * cycle_time
    throughput_mips = (instr / exec_time) / 1e6
    return {
        "config": config_name,
        "clock_ghz": freq / 1e9,
        "instr": instr,
```

Configuration	Clock (GHz)	CPI	Cycles (M)	Exec. Time (ms)	MIPS
Non-pipelined (single-cycle)	1.50	1.00	1.00	0.667	1500.0
Non-pipelined (multi-cycle)	2.00	4.00	4.00	2.000	500.0
5-stage pipeline (no hazard handling)	2.50	1.35	1.35	0.540	1851.9
5-stage pipeline (forwarding + stall)	2.50	1.15	1.15	0.460	2173.9
Superscalar OoO (2-issue, speculative)	3.00	0.60	0.60	0.200	5000.0
SMT (2-issue, 2 threads)	3.00	0.72	1.44	0.480	4166.7





14.1 Bottleneck identification

- The non-pipelined multi-cycle design is bottlenecked by its CPI (4.0) - every instruction pays for fetch, decode, execute, memory and write-back as separate serial cycles even though no other instruction is in flight to hide that latency.
- The unhandled 5-stage pipeline is bottlenecked by hazard stalls: CPI only falls to 1.35 despite pipelining because every dependent instruction still waits for its producer's write-back.
- The forwarding-enabled pipeline removes most of that penalty (CPI 1.15) but is still bounded by single-issue width - only one instruction can enter the pipeline per cycle no matter how independent the surrounding code is.
- The superscalar OoO design achieves the best CPI (0.60) and the highest throughput because it both pipelines and issues two independent instructions per cycle, but it is the most complex to build (reorder buffer, dependency tracking, speculation/recovery logic).
- SMT achieves a strong aggregate throughput (4166.7 MIPS, second only to the dedicated superscalar core) at a lower hardware cost than a full second execution core, but per-thread latency does not improve - it is a throughput-oriented, not a latency-oriented, technique.

15. Comparative Analysis and Recommended Configuration

Bringing together the results from Sections 9-13, the six configurations can be ranked as follows:

Rank	Configuration	Speedup vs baseline	Relative hardware cost
1	Superscalar OoO (2-issue, speculative)	10.00x	Highest
2	SMT (2-issue, 2 threads)	8.33x	High (shared with superscalar core)
3	5-stage pipeline (forwarding + stall)	4.35x	Moderate
4	5-stage pipeline (no hazard handling)	3.70x	Moderate
5	Non-pipelined (single-cycle control unit)	3.00x	Low
6	Non-pipelined (multi-cycle control unit)	1.00x (baseline)	Lowest

For raw arithmetic throughput and lowest CPI, the Superscalar Out-of-Order design with speculative execution is the most suitable configuration: it delivers the largest single-thread speedup (10.00x baseline) by combining pipelining, forwarding, out-of-order issue and branch speculation, and Section 7's arithmetic-algorithm results (Booth's multiplication and non-restoring division needing roughly half as many corrective operations as their naive counterparts) integrate naturally into such a design because a wider, out-of-order core can hide the algorithms' inherent multi-cycle latency behind independent work.

Where die area, power budget or design complexity rule out full superscalar OoO hardware, the practical recommendation is a forwarding-enabled 5-stage pipeline (4.35x baseline) as the minimum acceptable design - unhandled hazards alone cost 15% of that speedup (3.70x vs 4.35x), so hazard handling should never be treated as optional once a design is pipelined.

SMT is recommended as a complementary technique rather than a replacement for superscalar issue logic: it is most valuable in throughput-bound, multi-threaded workloads (e.g. server or batch workloads with many independent instruction streams) where its 8.33x aggregate baseline speedup is obtained largely by reusing already-built superscalar issue hardware rather than by duplicating an entire core, whereas single-thread latency-sensitive workloads benefit more from dedicating that same 2-issue hardware to OoO execution of one stream.

16. Conclusion

This project designed, implemented, analysed and evaluated an Integrated Processor Architecture Simulator that satisfies all twelve stated objectives. Working Python implementations were built and executed for: the CPU/register-file/ALU/memory/I/O functional-unit model and the fetch-decode-execute cycle with explicit register-transfer logging; signed integer addition and subtraction with overflow detection; the Carry Look-Ahead Adder, Booth's Multiplication Algorithm, and both restoring and non-restoring division; IEEE 754 single- and double-precision encoding and arithmetic; a 5-stage pipelined datapath exhibiting structural, data and control hazards; forwarding- and stall-based hazard handling with a measured 22% cycle reduction from forwarding alone; and superscalar out-of-order, speculative-execution and simultaneous-multithreading models that achieved 2.33x and 1.83x throughput improvements respectively over an in-order scalar baseline.

The final performance evaluation, using standard CPI / execution-time / throughput analysis across all six configurations on a common benchmark, showed a 10x overall speedup for the superscalar out-of-order design over a non-pipelined multi-cycle baseline, confirming that pipelining, hazard handling and instruction-level parallelism are complementary, cumulative techniques - each addresses a different source of lost performance, and the largest gains come from combining all of them rather than from any single technique in isolation. The Superscalar Out-of-Order configuration is therefore identified as the most suitable overall design for improved arithmetic processing, pipeline efficiency and processor performance, with SMT recommended as a cost-effective complement in throughput-oriented, multi-threaded environments.